

Math Foundations of Machine Learning

Neural Networks

Gabriel Wendell Celestino Rocha

July 2025

Table of Contents

Basics for Deep Learning

Neural Networks

- Sigmoid Neural Networks

- A Simple Network to Classify Handwritten Digits

Back-Propagation

- Notations

- The Cost Function

- The Four Fundamental Equations of Back-Propagation



Basics for Deep Learning

Conventional Machine Learning

- Limited in their ability to process data in their raw form.
- **Features!!**
 - **Coming up with features:** Difficult, time-consuming, requiring expert knowledge
 - **Tuning the features:** We spend a lot of times, before and during learning
- **Examples of features:** Histogram of Oriented Gradients (HOG), the scale-invariant feature transform (SIFT) (Lowe, 1999), etc.

Representation Learning

- **Discover representations, automatically**
⇒ The machine is fed with **raw data**
- **Deep Learning:** methods are representation-learning methods with multiple levels of **representation/abstraction**
 - Simple non-linear modules ⇒ higher and abstract representation
 - With the composition of enough such transformations, very complex functions can be learned
- **Key Aspects**
 - **Layers of features** are not designed by human engineers
 - **Learn features** from data using a general-purpose learning procedure

Advances in Deep Learning

- Image recognition
- Speech recognition
- Natural language understanding:
 - Machine translation
 - Image 2 text
 - Sentiment analysis
 - **Question-answering (QA) machine:** [IBM's Watson](#), 2011, defeated legendary Jeopardy champions Brad Rutter and Ken Jennings, winning the first place prize of \$1 million.
- Many other domains
 - Predicting the activity of potential drug molecules
 - Analyzing particle accelerator data
 - Reconstructing brain circuits
 - Predicting the effects of mutations in noncoding DNA on gene expression and disease
- **Image-based Classifications:** Deep learning has provided break through results in speech recognition and image classification.

Why/What about Deep Learning?

- **Why is it generally better than other methods** on image, speech, and certain other types of data? Short answers:
 - Deep learning means using a **neural network** with **several layers of nodes** between input and output
 - The series of layers between input & output do **feature identification and processing** in a series of stages, just as our brains seem to do.
- Multi-layer neural networks have been more than 30 years (Rina Dechter, 1986). **What is actually new?**
 - We have always had good algorithms for learning the weights in networks **with 1 hidden layer**.
 - **The New** are: methods for training many-layer networks.

AI vs ML vs Deep Learning

- **Artificial Intelligence (AI)**: Intelligence exhibited by machines
- **Machine Learning (ML)**: An approach to achieve AI
- **Deep Learning (DL)**: A technique for implementing ML
 - Feature/Representation-learning
 - Multi-layer neural networks (NN)
 - Back-propagation
(In the 1980s and 1990s, researchers did not have much luck, except for a few special architectures.)
 - **New ideas** enable learning in deep NNs, since 2006

Back-propagation to Train Multi-layer Architectures

- Nothing more than a **practical application of the chain rule**, for derivatives
- Forsaken because poor **local minima**
- **Revived around 2006** by unsupervised learning procedures with unlabeled data
 - **CIFAR** ((Canadian Institute for Advanced Research)
 - Recognizing handwritten digits or detecting pedestrians
 - Speech recognition by **GPUs**, with 10 or 20 times faster (Raina *et al.*, 2009) and (Bengio, 2013)
 - Local minima become rarely a problem
- **Convolutional Neural Networks (CNN)**
 - Widely adopted by computer-vision community (LeCun *et al.*, 1989)
- **Activations**
 - Non-linear functions: $\max(z, 0)$ (**ReLU**), $\tanh(z)$, $1 / (1 + e^{-z})$

Machine Learning Challenges We've Yet to Overcome

- **Interpretability:** Although ML has come very far, researchers still **don't know exactly how deep training nets work**.
 - If we don't know how training nets actually work
 - ⇒ **How do we make any real progress?**
- **One-Shot Learning:** We still haven't been able to achieve one-shot learning. **Traditional networks need a huge amount of data**, and are often in the form of **extensive iterative training**.
 - Instead, we should find a way to enable neural networks to learn, *using just a few examples*.
 - Current neural networks are **gradient-and-iteration-based**
 - ⇒ **Can we modify/replace it?**



Neural Networks

Neural Networks

Recall: In 1957, Frank Rosenblatt invented the **perceptron** algorithm:

- For input values: $\mathbf{x} = (x_1, x_2, \dots, x_d)^T$,
- Learn weight vector: $\mathbf{w} = (w_1, w_2, \dots, w_d)^T$,
- Get the net input: $z = w_1x_1 + w_2x_2 + \dots + w_dx_d = \mathbf{w} \cdot \mathbf{x}$,
- Classify, using the activation function:

$$\phi(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ -1 & \text{otherwise} \end{cases}, \quad z = \mathbf{w} \cdot \mathbf{x},$$

or, equivalently,

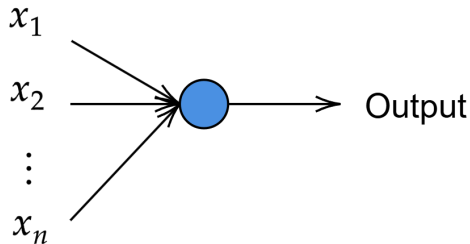
$$\phi(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{otherwise} \end{cases}, \quad z = b + \mathbf{w} \cdot \mathbf{x},$$

where $b = -\theta$ is the **bias**.

Neural Networks

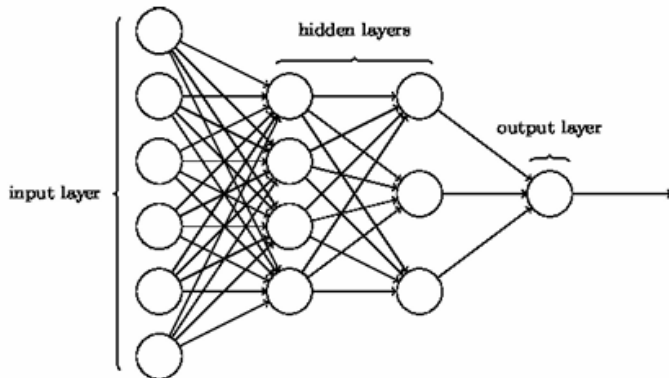
Perceptron is the simplest artificial neuron:

- It **makes decisions** by **weighting up evidence**
- However, it is **not a complete model** for decision-making!



Complex Networks of Perceptrons

- **Perceptrons as a building block:**
 - What the example illustrates is **how a perceptron can weigh up** different kinds of evidence in order to make decisions



- It should seem plausible that a **complex network of perceptrons** could make quite subtle decisions

An Issue on Perceptron Networks

- **Thresholding:** A small change in the weights or bias of any single perceptron in the network can sometimes cause the output of that perceptron to **completely flip**, say from -1 to 1 .
 - That flip may **then cause the behavior of the rest of the network to completely change** in some very complicated way.
- We can over come this problem by introducing a new type of artificial neuron, e.g., a **sigmoid neuron**.

Sigmoid Neural Networks

Logistic Sigmoid Function

The **logistic sigmoid function** is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- They are built with **sigmoid neurons**
- The output of a sigmoid neuron with inputs $\mathbf{x}$, weights $\mathbf{w}$, and bias b is:

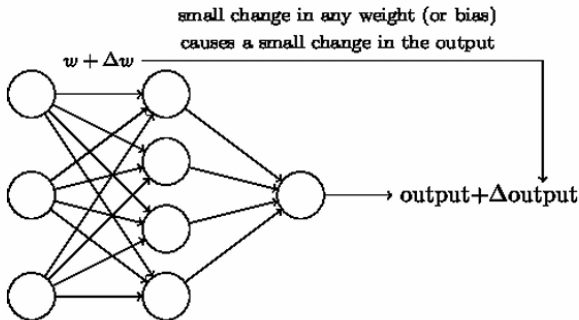
$$\sigma(z) = \frac{1}{1 + \exp(-b - \mathbf{w} \cdot \mathbf{x})}.$$

which we considered as the **logistic regression** model.

Sigmoid Neural Networks

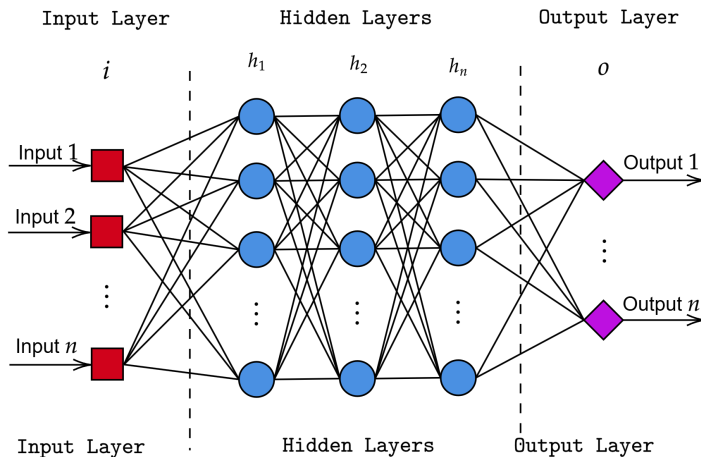
- Advantages of the sigmoid activation:
 - It allows **calculus** to design learning rules [$\sigma' = \sigma(1 - \sigma)$]
 - Small changes in weights and bias produce a corresponding small change in the output.

$$\Delta(\text{output}) \approx \sum_j \frac{\partial(\text{output})}{\partial w_j} \Delta w_j + \frac{\partial(\text{output})}{\partial b} \Delta b$$



The Architecture of (Smooth) Neural Networks

- The leftmost layer is called the **input layer**, and the neurons within the layer are called **input neurons**.
- The right most layer is the **output layer**.
- The middle layers are called **hidden layers**.



The Architecture of (Smooth) Neural Networks

- **The design of the input and output layers in a network is often straightforward.** For example, for the classification of handwritten digits:
 - If the images are in 28×28 grayscale pixels, then we'd have 784 ($= 28 \times 28$) input neurons.
 - It is heuristic to set 10 neurons in the output layer. (rather than 4, where $2^4 = 16 \geq 10$)
- There can be **quite an art to the design of the hidden layers.**
 - In particular, **it is not possible to sum up the design process for the hidden layers with a few simple rules of thumb.**
 - Instead, neural networks researchers have developed **many design heuristics** for the hidden layers, which help people get the behavior they want out of their nets.
 - For example, **such heuristics** can be used to help determine how to trade off the **number of hidden layers** against **the accuracy and the time** required to train the network.

A Simple Network to Classify Handwritten Digits

- The problem of recognizing handwritten digits has two components: **segmentation** and **classification**.

504192 $\Rightarrow$

5	0	4	1	9	2
---	---	---	---	---	---

- We'll focus on algorithmic components for the classification of individual digits.

MNIST Dataset

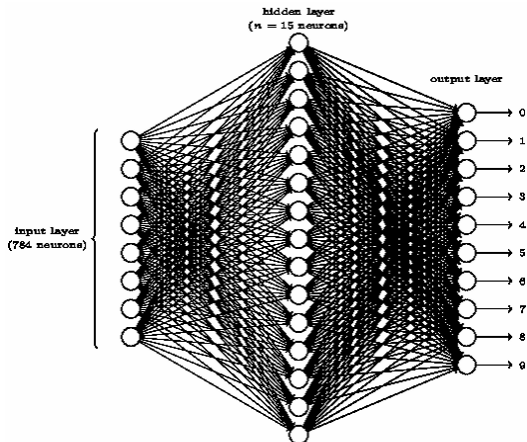
A modified subset of two datasets collected by NIST (US National Institute of Standards and Technology):

- The first part contains 60,000 images (for training)
- The second part is 10,000 images (for test)

Each image is in 28×28 grayscale pixels

A Simple Feed-Forward Neural Network

- Let's concentrate on **the first output neuron**, the one that is trying to decide whether or not the input digit is a 0.
- It does this by **weighing up evidence** from the hidden layer of neurons.



What the Neural Network Will Do?

- What are those hidden neurons doing?

- Let's suppose for the sake of argument that **the first neuron** in the hidden layer may detect whether or not an image like the following is present



It can do this by **heavily weighting input pixels** which overlap with the image, and only lightly weighting the other inputs.

- Similarly, let's suppose that **the second, third, and fourth neurons** in the hidden layer detect whether or not the following images are present



- As you may have guessed, these four images together make up the 0 image that we saw in the line of digits



- So if **all four of these hidden neurons are firing**, then we can conclude that the digit is a 0.

Learning with Gradient Descent

- **Dataset:** $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}, i = 1, 2, \dots, N$.
 - $\mathbf{y}^{(i)}$? For example, if an image $\mathbf{x}^{(i)}$ depicts a 2, then:

$$\mathbf{y}^{(i)} = (0, 0, 1, 0, 0, 0, 0, 0, 0)^T$$

- **Cost Function**

$$C(\mathbf{W}, B) = \frac{1}{2N} \sum_i \left\| \mathbf{y}^{(i)} - \mathbf{a}(\mathbf{x}^{(i)}) \right\|^2,$$

where $\mathbf{W}$ denotes the collection of all weights in the network, B all the biases, and $\mathbf{a}(\mathbf{x}^{(i)})$ is the vector of outputs from the network when $\mathbf{x}^{(i)}$ is input.

- **Gradient Descent Method**

$$\begin{bmatrix} \mathbf{W} \\ B \end{bmatrix} \leftarrow \begin{bmatrix} \mathbf{W} \\ B \end{bmatrix} + \begin{bmatrix} \Delta \mathbf{W} \\ \Delta B \end{bmatrix} = \begin{bmatrix} \mathbf{W} \\ B \end{bmatrix} - \eta \begin{bmatrix} \nabla_{\mathbf{W}} C \\ \nabla_B C \end{bmatrix}$$

Learning with Gradient Descent

Note about the Gradient Descent Method

To compute the gradient ∇C , we need to compute the gradients $\nabla C_{\mathbf{x}^{(i)}}$ separately for each training input, $\mathbf{x}^{(i)}$, and then average them:

$$\nabla C = \frac{1}{N} \sum_i \nabla C_{\mathbf{x}^{(i)}} .$$

- Unfortunately, when the number of training inputs is very large, it can take a long time, and learning thus occurs slowly.
- An idea called **stochastic gradient descent** can be used to speed up learning.

Stochastic Gradient Descent

The idea is to estimate the gradient ∇C by computing $\nabla C_{\mathbf{x}^{(i)}}$ for a **small sample of randomly chosen training inputs**. By averaging over this small sample, it turns out that we can quickly get a good estimate of the true gradient ∇C ; this helps speed up gradient descent, and thus learning.

- Pick out a small number of randomly chosen training inputs ($m \gg N$):

$$\tilde{\mathbf{x}}^{(1)}, \tilde{\mathbf{x}}^{(2)}, \dots, \tilde{\mathbf{x}}^{(m)},$$

which we refer to as a **mini-batch**.

- Average $\nabla C_{\tilde{\mathbf{x}}^{(k)}}$ to approximate the gradient ∇C . That is,

$$\frac{1}{m} \sum_{k=1}^m \nabla C_{\tilde{\mathbf{x}}^{(k)}} \approx \nabla C \stackrel{\text{def}}{=} \frac{1}{N} \sum_i \nabla C_{\mathbf{x}^{(i)}}.$$

- For classification of handwritten digits for the MNIST dataset, you may choose: `batch_size = 10`.

Stochastic Gradient Descent

Note about the Stochastic Gradient Descent Method

In practice, you can implement the stochastic gradient descent as follows.
For an epoch:

- Shuffle the dataset
- For each m samples (selected from the beginning), update $(\mathbf{W}, B)$ using the approximate gradient.

$$\frac{1}{m} \sum_{k=1}^m \nabla C_{\tilde{\mathbf{x}}^{(k)}} \approx \nabla C \stackrel{\text{def}}{=} \frac{1}{N} \sum_i \nabla C_{\mathbf{x}^{(i)}} .$$

Accuracy Comparison

- `scikit-learn`'s SVM classifier using the default settings: 9435/10000
- A well-tuned SVM $\approx 98.5\%$
- Well-designed (convolutional) NN: 9979/10000 (only 21 missed!)

Note about the Performance

For **well-designed neural networks**, the performance is close to **human-equivalent**, and is **arguably better**, since quite a few of the MNIST images are difficult even for humans to recognize with confidence, e.g.



Moral of the Neural Networks

- Let all the complexity be learned, automatically, from data
- Simple algorithms can perform well for some problems

(sophisticated algorithm) $\leq$ ((simple learning algorithm + good training data)



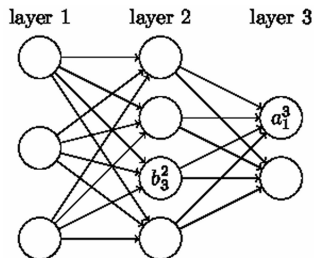
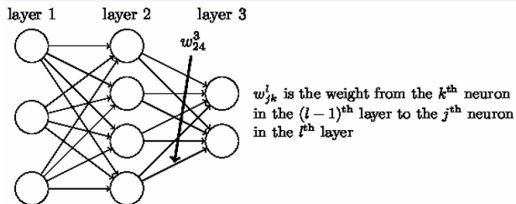
Back-Propagation

Back-Propagation

- In the previous section, we saw an example of neural networks that could learn their weights and biases using the stochastic gradient descent algorithm.
- In this section, we will see **how to compute the gradient**, more precisely, **the derivatives of the cost function** with respect to weights and biases **in all layers**.
- *The back-propagation is a practical application of the chain rule* for the computation of derivatives.
- The back-propagation algorithm was originally introduced in the 1970s, but its importance was not fully appreciated until a famous 1986 paper by Rumelhart-Hinton-Williams, in *Nature*.

Notations

- Let's begin with notations which let us refer to weights, biases, and activations in the network in an unambiguous way.
 - w_{jk}^ℓ : the **weight** for the connection from the k^{th} neuron in the $(\ell - 1)^{\text{th}}$ layer to the j^{th} neuron in the ℓ^{th} layer
 - b_j^ℓ : the **bias** of the j^{th} neuron in the ℓ^{th} layer
 - a_j^ℓ : the **activation** of the j^{th} neuron in the ℓ^{th} layer.



- With these notations, the activation a_j^ℓ reads to:

$$a_j^\ell = \sigma \left(\sum_k w_{jk}^\ell a_k^{\ell-1} + b_j^\ell \right) ,$$

where the sum is over all neurons k in the $(\ell - 1)^{\text{th}}$ layer. Denote the **weighted input** by:

$$z_j^\ell := \sum_k w_{jk}^\ell a_k^{\ell-1} + b_j^\ell .$$

- Now, define:
 - $W^\ell = [w_{jk}^\ell]$: the weight matrix for layer ℓ
 - $\mathbf{z}^\ell = [z_j^\ell]$: the weighted input vector for layer ℓ
 - $\mathbf{b}^\ell = [b_j^\ell]$: the bias vector for layer ℓ
 - $\mathbf{a}^\ell = [a_j^\ell]$: the activation vector for layer ℓ
- Then, the activation function can be rewritten (in a vector form) as:

$$\mathbf{a}^\ell = \sigma(\mathbf{z}^\ell) = \sigma \left(W^\ell \mathbf{a}^{\ell-1} + \mathbf{b}^\ell \right) .$$

The Cost Function

The Cost Function

With the notations, the quadratic cost function has the form

$$C = \frac{1}{2N} \sum_{\mathbf{x}} \left\| \mathbf{y}(\mathbf{x}) - \mathbf{a}^L(\mathbf{x}) \right\|^2 ,$$

where N is the total number of training examples, $\mathbf{y}(\mathbf{x})$ is the corresponding desired output for the training example $\mathbf{x}$, and L denotes the number of layers in the network.

Two Assumptions for the Cost Function

1. The cost function can be written as an average $C = \frac{1}{N} \sum_{\mathbf{x}} C_{\mathbf{x}}$, over cost functions $C_{\mathbf{x}}$ for individual training examples $\mathbf{x}$.
2. The cost function can be written as a function of the outputs from the neural network ($\mathbf{a}^L$).

The Cost Function

Remark on Cost Functions

Thus the cost function satisfies the assumptions, with

$$C_{\mathbf{x}} = \frac{1}{2} \left\| \mathbf{y}(\mathbf{x}) - \mathbf{a}^L(\mathbf{x}) \right\|^2 = \frac{1}{2} \sum_j \left[y_j(\mathbf{x}) - a_j^L(\mathbf{x}) \right]^2 .$$

- The reason we need the first assumption is because what the back-propagation actually lets us do is **compute the partial derivatives $\partial C_{\mathbf{x}} / \partial w_{jk}^{\ell}$ and $\partial C_{\mathbf{x}} / \partial b_j^{\ell}$ for a single training example.**
 - We then **can recover $\partial C / \partial w_{jk}^{\ell}$ and $\partial C / \partial b_j^{\ell}$ by averaging over training examples.**
- With the assumptions in mind, we may **focus on computing the partial derivatives for a single example.**

The Four Fundamental Equations of Back-Propagation

The back-propagation is about understanding *how* **changing the weights and biases in a network changes the cost function**, which means computing the partial derivatives $\partial C / \partial w_{jk}^\ell$ and $\partial C / \partial b_j^\ell$.

Learning Error

We define the **learning error** (or, **error**) of neuron j in layer ℓ by:

$$\delta_j^\ell := \frac{\partial C}{\partial z_j^\ell} .$$

The back-propagation will give us a way of computing $\boldsymbol{\delta}^\ell = [\delta_j^\ell]$ for every layer ℓ , and then relating those errors to the quantities of real interest, $\partial C / \partial w_{jk}^\ell$ and $\partial C / \partial b_j^\ell$.

The Four Fundamental Equations of Back-Propagation

Theorem: Fundamental Equations of Back-Propagation

Suppose that the cost function C satisfies the two assumptions presented above so that it represents the cost for a single training example. Assume the network contains L layers, of which the feed-forward model is given as in:

$$a_j^\ell = \sigma(z_j^\ell) \quad , \quad z_j^\ell = \sum_k w_{jk}^\ell a_k^{\ell-1} + b_j^\ell \quad , \quad \ell = 2, 3, \dots, L.$$

Then,

$$\delta_j^\ell = \frac{\partial C}{\partial a_j^\ell} \sigma'(z_j^\ell) \quad , \quad \frac{\partial C}{\partial b_j^\ell} = \delta_j^\ell \quad , \quad \ell = 2, \dots, L,$$

$$\delta_j^\ell = \sum_k w_{kj}^{\ell+1} \delta_k^{\ell+1} \sigma'(z_j^\ell) \quad , \quad \ell \in [L-1, 2] \quad , \quad \frac{\partial C}{\partial w_{jk}^\ell} = a_k^{\ell-1} \delta_j^\ell \quad , \quad \ell = 2, \dots, L.$$

The Hadamard Product / Schur Product

Hadamard Product / Schur Product

A frequently used algebraic operation is the **element-wise product** of two vectors/matrices, which is called the **Hadamard product** or the **Schur product**, and defined as

$$(\mathbf{c} \odot \mathbf{d})_j = c_j d_j .$$

- For example,

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} \odot \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 \cdot 3 \\ 2 \cdot 4 \end{bmatrix} = \begin{bmatrix} 3 \\ 8 \end{bmatrix} .$$

- In Numpy, $\mathbf{A} * \mathbf{B}$ denotes the Hadamard product of $\mathbf{A}$ and $\mathbf{B}$, while $\mathbf{A} . \text{dot} (\mathbf{B})$ or $\mathbf{A} @ \mathbf{B}$ produces the regular matrix-matrix multiplication.

The Hadamard Product / Schur Product

Remark on the Hadamard Product

The four fundamental equations can be written in a vector form as:

$$\begin{cases} \delta^L = \nabla_{\mathbf{a}^L} C \odot \sigma'(\mathbf{z}^L) , \\ \delta^\ell = \left[(\mathbf{W}^{\ell+1})^T \delta^{\ell+1} \right] \odot \sigma'(\mathbf{z}^\ell) , & \ell = L-1, \dots, 2 , \\ \nabla_{\mathbf{b}^\ell} C = \delta^\ell , & \ell = 2, \dots, L , \\ \nabla_{\mathbf{W}^\ell} C = \delta^\ell (\mathbf{a}^{\ell-1})^T , & \ell = 2, \dots, L , \end{cases}$$

Let's now summarize the **back-propagation** algorithm.

The Back-Propagation Algorithm

1. **Input $\mathbf{x}$:** Set the corresponding activation $\mathbf{a}^1$ for the input layer.

2. **Feed-forward:**

$$\mathbf{z}^\ell = \mathbf{W}^\ell \mathbf{a}^{\ell-1} + \mathbf{b}^\ell \quad , \quad \mathbf{a}^\ell = \sigma(\mathbf{z}^\ell) \quad , \quad \ell = 2, 3, \dots, L \quad ,$$

3. **Output error δ^L :**

$$\delta^L = \nabla_{\mathbf{a}^L} C \odot \sigma'(\mathbf{z}^L) \quad ,$$

4. **Back-propagate the error:**

$$\delta^\ell = \left[\left(\mathbf{W}^{\ell+1} \right)^T \delta^{\ell+1} \right] \odot \sigma'(\mathbf{z}^\ell) \quad , \quad \ell = L - 1, \dots, 2 \quad .$$

5. **The gradient of the cost function:**

$$\nabla_{\mathbf{b}^\ell} C = \delta^\ell \quad , \quad \nabla_{\mathbf{W}^\ell} C = \delta^\ell \left(\mathbf{a}^{\ell-1} \right)^T \quad , \quad \ell = 2, \dots, L \quad .$$

An SDG Learning Step, Based on a Mini-Batch

1. Input a set of training examples of size m ;
2. **Initialize:** $\Delta W = 0$ and $\Delta B = 0$;
3. For each training example $\mathbf{x}$:
 - (a) Apply the back-propagation algorithm to find

$$\nabla_{\mathbf{b}^\ell} C_{\mathbf{x}} = \boldsymbol{\delta}^{\mathbf{x},\ell} \quad , \quad \nabla_{\mathbf{W}^\ell} C_{\mathbf{x}} = \boldsymbol{\delta}^{\mathbf{x},\ell} \left(\mathbf{a}^{\mathbf{x},\ell-1} \right)^T \quad , \quad \ell = 2, \dots, L \quad .$$

- (b) Update the gradient:

$$\begin{cases} \Delta B = \Delta B + [\nabla_{\mathbf{b}^2} C_{\mathbf{x}}] \cdots [\nabla_{\mathbf{b}^L} C_{\mathbf{x}}] & , \\ \Delta W = \Delta W + [\nabla_{\mathbf{W}^2} C_{\mathbf{x}}] \cdots [\nabla_{\mathbf{W}^L} C_{\mathbf{x}}] & . \end{cases}$$

4. **Gradient descent:** Update the biases and weights:

$$B = B - \frac{\eta}{m} \Delta B \quad , \quad W = W - \frac{\eta}{m} \Delta W \quad .$$

Saturated Neurons

- The four fundamental equations satisfy independently of choices of **the cost function C and the activation σ** .
- A consequence of the fourth vector form of the four fundamental equations of back-propagation is that is $\mathbf{a}^{\ell-1}$ is small (in modulus), gradient term $\partial C / \partial W^\ell$ will also tend to be small. In this case, we'll say the weight **learns slowly**, meaning that it's not changing much during gradient descent.
 - In other words, a consequence of that equation is that **weights output from low-activation neurons learn slowly**.
 - The sigmoid function σ becomes very flat when $\sigma(z_j^L)$ is approximately 0 or 1. When this occurs we will have $\sigma'(z_j^L) \approx 0$. So, a weight in the final layer will learn slowly if the output neuron is either low activation (≈ 0) or high activation (≈ 1). In this case, we usually say **the output neuron has saturated** and, as a result, the weight is learning slowly (or stopped).
- Similar remarks hold also in other layers and for the biases as well.

Saturated Neurons

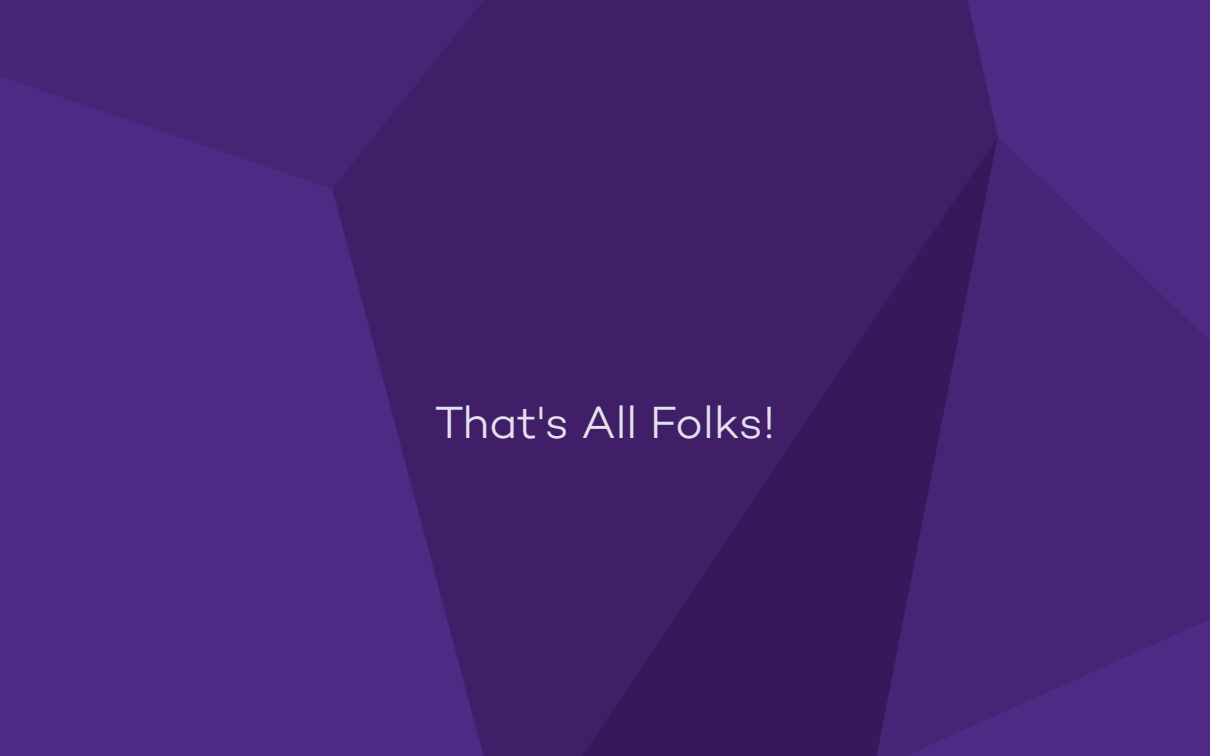
Summing Up

- Weights and biases will learn slowly if:
 - Either **the in-neurons** (upwind) are in low-activation.
 - Or **the out-neurons** (downwind) have saturated.

Designing Activation Functions

The four fundamental equations can be used to design activation functions which have **particular desired learning properties**.

- For example, **suppose we were to choose a (non-sigmoid) activation function σ so that σ' is always positive, and never gets close to zero.**
 - **That would prevent the slow-down of learning** that occurs when ordinary sigmoid neurons saturate.
- **Learning accuracy and efficiency** can be improved by finding **more effective cost and activation functions**.

The background is an abstract composition of several overlapping triangles in various shades of purple, ranging from a deep, dark purple to a lighter, more vibrant purple. The triangles are arranged in a way that creates a sense of depth and geometric complexity.

That's All Folks!